

LESSON GUIDE

Technology Plus

Unit 2: Programming

4.3.4 – Objects

Lesson Overview:

In this lesson, students will expand their knowledge of object-oriented programming by working with classes to model real-world objects.

Students will:

- Explain the key components of objects.
- Create mock classes to represent objects.

Guiding Question: How can we model real-world objects in a programming environment?

Suggested Grade Levels: 8-12

Technology Needed: No

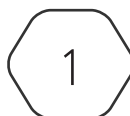
Approximate Time Required: 50-70 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 4.3 – Explain the purpose and use of programming concepts.
 - Identifiers
 - Variables
 - Constants
 - Arrays
 - Functions
 - **Objects**
 - **Properties**
 - **Attributes**
 - **Methods**

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).



CYBER.ORG
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center
All Rights Reserved. Not for Distribution.

Lesson Background

Background Information:

In 4.1.1, students were briefly introduced to object-oriented programming (OOP) languages. This lesson will focus heavily on the use of classes to model objects with common properties, attributes, and methods. Students will then apply the knowledge from this lesson in lesson 4.4.1 – Programming Language Categories.

While programming the classes is outside the scope of Technology Plus, you might consider demonstrating class creation in the OOP language of your choice (Python, Java, C++, etc.).

Materials Needed:

Materials per Class	• Lesson Guide • Lesson Slides 4.3.4 – Objects • Activity Slides 4.3.4 – Mapping Objects
Materials per Group	• Activity Handout 4.3.4 – Mapping Objects
Materials per Student	• Student Notes 4.3.4 – Objects • Student Handout 4.3.4 – Objects • Assessment 4.3.4 – Objects (optional)

Lesson Background

Cyber Terms:

Term	Definition
object	an instance of a class; the most basic building block of object-oriented programming made up of properties, attributes, and methods
instance	a concrete version of the class with initialized values and data
class	a blueprint or template used to create objects
property	a description of characteristics of an object; the state of an object
attribute	a description of properties – provides the actual value or data for a property
method	actions, or functions, that an object can do; the behaviors of an object
encapsulation	bundling data and methods together, as fields, to promote modularity and organization

4.3.4 – Objects

Guiding Question: How can we model real-world objects in a programming environment?

Lesson Launch (5-10 minutes)

- Discuss the guiding question and lesson objectives.
- Slide 3 is animated to guide students through the following thought exercise.
- **Ask** students to close their eyes and picture a car.
- **Ask:** Does the car have doors? How many? What color is the exterior? The interior?
- Students should open their eyes, and you will reveal the different car icons they may have been picturing.
- **Ask** students to share what they pictured.
- Consider recording answers from multiple students for reference throughout the lesson.

Objects as a Programming Tool (15-20 minutes)

Objects

- Distribute Student Handout 4.3.4 – Objects and encourage students to fill in the answers during the lesson.
- Recall lesson 4.1.1 and the quick introduction to Object-Oriented Programming (OOP) as a paradigm used to organize programming projects by creating classes that model real-world objects.
- **Objects** are the specific instances we build from a class. Each object requires an identity, or name.
- Each identity initializes properties for that **instance** of the class - the object.
- Instantiation is the process of creating a new instance, a concrete version, of a class, assigning it an identity (a named variable), and setting initial values for properties.
- Once we have an object, we can call methods from the class.
- Example:
 - `Car myCar = new Car();`
 - Creates a new variable of type Car, called myCar, and initializes it to a new Car object.

Classes

- A **class** is a template for how similar objects will be created based on properties and methods.
- **Encapsulation** is the process of bundling data and methods together to promote modularity and organization. In OOP, this is why we use Classes, to create frameworks for common objects we plan to reuse.
- Explain that we will view the creation of a sample class for a car object. While students may not be familiar with Java syntax, it should be understandable. If you teach another language, consider

replacing the code snippets with your own example.

- Explain the updated instance call to the Car class.

Properties and Attributes

- Explain that while properties and attributes have some nuanced differences, it is common to use them interchangeably in programming. Provide the definitions:
 - A **property** is a description of a characteristic of an object. For example, a Human object has a property of eye color.
 - An **attribute** is a description of a property. For example, an instance of a Human object named Jeff has an attribute of 'blue' for the eye color property.
- For most of the lesson, we'll use the words as synonyms.
- Direct students back to the Car class to discuss the properties/attributes of a car.
- Explain the sample properties added to the class and show the updated calls to the Car class.

Methods

- Explain that **methods**, also often referred to as functions, are the actions an object can do.
- Methods are referred to as "getters" and "setters". A "getter" is a method that retrieves a property from the object. A "setter" is a method that modifies a property. Getters return data. Setters do not return data; they just update data.
- Direct students back to the Car class again to discuss methods for a car.
- Explain the sample methods added and show the updated calls to the Car class.

Activity: Mapping Objects (20-30 minutes)

- Distribute Activity Handout 4.3.4 – Mapping Objects.
- Slide 2 is animated to promote review prior to breaking into groups and starting the activity.
- Consider whether you want students to work independently, in pairs, or in groups. Hide or delete the other slides. Also assign the expected number of properties and methods for each object. Most examples provide 3, but you can adjust for consideration of your students and overall time available.
- While students are working, walk around and answer questions as appropriate.
- Decide whether you want to collect work or review solutions as a class.

Object	Properties	Methods
A: Lamp	color, brightness, isOn, height	turnOn(), turnOff(), changeBrightness()
B: Refrigerator	color, temperature, isOpen, items	open(), close(), addItem(), removeItem()
C: Chair	material, height, hasArmrest, hasBack	adjustHeight(), rotate()

D: Notebook	color, subject, numberOfPages, currentPage	turnPage(), writeNote(), tearPage()
E: Backpack	color, capacity, contents	addItem(), removeItem(), zip()
F: Pencil	length, isSharp, type	sharpen(), write(), erase()
G: Textbook	title, author, pageCount, subject	open(), turnPage(), highlightPassage()
H: Phone	color, brand, apps, cameras, batteryLevel, isSmart	makeCall(), installApp(), charge(), takePhoto()
I: Computer	type, batteryLevel, isOn, brand	turnOn(), turnOff(), charge()
J: Bicycle	color, gear, speed	pedal(), brake(), changeGear()
K: Playlist	genre, songCount, isPublic	addSong(), deleteSong(), play(), pause()
L: Restaurant	cuisine, openTables, waitstaff	cook(), serve(), seatGuests()

Putting It All Together (5 minutes)

- Review what students learned today by answering the questions below as a class.
- What is a benefit of a class?
It creates a blueprint for objects of similar types. It allows us to model real-world objects and reuse the code over and over.
- Describe the differences between a property and a method.
A property describes characteristics of an object (nouns) and a method describes behaviors of an object (actions).
- What is encapsulation and how does it relate to objects?
Encapsulation is the process of organizing data in bundles of similar features and functions, the way we create classes to organize the properties and methods of an object.
- Create a sample object and define two sample properties and two sample methods for it.
Regular Polygon
Properties: number of sides, length of sides
Methods: reflect-y(), rotate-90(), dilate()